# Bab 1 Arsitektur Aplikasi Web Modern dan Lingkungan Pengembangan Lanjut

## Tujuan Pembelajaran

Setelah mempelajari bab ini, Anda mampu:

1. Menjelaskan perbedaan *server-side rendering* (SSR), *single-page application* (SPA), dan pendekatan SSR dengan interaktivitas komponen.
2. Membandingkan monolith, modular monolith, dan microservices berdasarkan kebutuhan domain, ukuran tim, pola penerapan, serta kompleksitas operasional.
3. Menjelaskan peran *web framework*, pola *Model–View–Controller* (MVC), dan *service container* dalam pengembangan aplikasi Laravel.
4. Menyiapkan lingkungan pengembangan aplikasi Laravel dengan manajemen dependensi, basis data MariaDB, Redis, Git, dan *quality gate*.
5. Menganalisis keputusan arsitektur dan konfigurasi awal untuk studi kasus aplikasi Kantin Multi-Tenant.

**Peta bab.** Bab ini menghubungkan konsep arsitektur dengan pembangunan proyek Kantin Multi-Tenant. Contoh konfigurasi, data numerik, diagram, dan hasil perhitungan dalam bab ini bersifat *ilustratif* kecuali dinyatakan lain. Seluruh kutipan kode telah diselaraskan dengan repositori proyek *Kantin Multi-Tenant* (Laravel 13) dan *Modul Praktikum* yang menyertainya.

## 1.1 Mengapa Arsitektur dan Lingkungan Pengembangan Penting

Aplikasi web modern jarang gagal hanya karena satu fungsi tidak berjalan. Masalah yang lebih sering muncul adalah kode sulit diubah, fitur baru merusak fitur lama, konfigurasi setiap anggota tim berbeda, atau aplikasi berjalan di komputer pengembang tetapi gagal saat dipindahkan ke lingkungan lain.

Arsitektur aplikasi membantu Anda membagi tanggung jawab antara antarmuka, aturan bisnis, data, dan layanan pendukung. Sementara itu, lingkungan pengembangan membantu seluruh anggota tim bekerja dengan versi perangkat, dependensi, dan standar kualitas yang sama.

Dalam bab ini, kita membahas dua keputusan awal yang saling berkaitan. Pertama, bagaimana aplikasi disusun dan dirender kepada pengguna. Kedua, bagaimana tim menyiapkan lingkungan kerja agar aplikasi dapat dibangun, diuji, dan dikembangkan secara konsisten.

Studi kasus yang digunakan adalah aplikasi Kantin Multi-Tenant. Aplikasi semacam ini melibatkan beberapa peran, misalnya administrator, pengelola tenant, pelanggan, dan petugas dapur. Karena fitur-fitur tersebut saling berhubungan, struktur aplikasi perlu dirancang sejak awal agar tidak berubah menjadi kumpulan kode yang sulit dipelihara.

## 1.2 Memahami Pilihan Rendering Web

### 1.2.1 Server-side rendering

*Server-side rendering* atau SSR adalah pendekatan ketika server membentuk halaman HTML sebelum mengirimkannya ke peramban. Ketika pengguna membuka sebuah halaman, server

memproses permintaan, mengambil data yang diperlukan, menyusun HTML, lalu mengirimkan hasilnya sebagai respons.

Pada SSR klasik, sebagian besar interaksi pengguna memicu permintaan baru ke server dan pemuatan ulang halaman. Pendekatan ini relatif mudah dipahami karena alur kerja utamanya berada di server.

Contoh sederhana adalah halaman daftar menu. Pengguna membuka alamat `/menu`, server mengambil data menu dari basis data, lalu mengirim halaman HTML yang berisi daftar menu tersebut.

SSR sering sesuai untuk aplikasi yang:

- Mengutamakan halaman yang dapat langsung ditampilkan tanpa memuat JavaScript dalam jumlah besar.
- Memiliki banyak halaman berbasis formulir atau konten.
- Membutuhkan struktur halaman yang mudah dirender oleh peramban dan mesin pencari.
- Dikerjakan oleh tim yang lebih nyaman dengan alur pengembangan berbasis server.

Namun, SSR tidak otomatis selalu lebih cepat. Waktu tampil halaman tetap dipengaruhi oleh waktu respons server, kueri basis data, *caching*, ukuran HTML, gambar, CSS, dan kondisi jaringan pengguna.

### 1.2.2 Single-page application

*Single-page application* atau SPA adalah aplikasi web yang memuat kerangka halaman di awal, kemudian memperbarui bagian antarmuka melalui JavaScript tanpa memuat ulang seluruh halaman. Server biasanya menyediakan *Application Programming Interface* (API) untuk mengirim dan menerima data, sering kali dalam format JSON.

Sebagai contoh, ketika pengguna menambahkan menu ke keranjang pada SPA, halaman tidak harus dimuat ulang sepenuhnya. JavaScript dapat mengirim permintaan ke API, menerima data keranjang terbaru, lalu memperbarui ikon atau jumlah item pada keranjang.

SPA dapat memberikan pengalaman interaksi yang responsif, terutama untuk antarmuka dengan banyak perubahan pada sisi pengguna. Namun, pendekatan ini juga menambah kebutuhan pengelolaan *state*, perutean di klien, ukuran berkas JavaScript, penanganan kesalahan API, dan strategi pengamanan antarmuka.

> **Catatan.** SPA tidak selalu lebih lambat daripada SSR. Namun, SPA berisiko memiliki waktu muat awal yang lebih tinggi apabila berkas JavaScript besar, jaringan pengguna lambat, atau strategi pemecahan berkas (*code splitting*) tidak diterapkan dengan baik.

### 1.2.3 SSR dengan interaktivitas komponen

Pendekatan ketiga menggabungkan render awal di server dengan interaktivitas pada bagian tertentu dari halaman. Dalam konteks Laravel, Blade dapat digunakan untuk membangun halaman dari server, sedangkan Livewire dapat menambahkan interaksi komponen tanpa mengharuskan pengembang membangun SPA penuh.

Misalnya, halaman katalog menu dapat dirender pertama kali oleh Blade. Ketika pengguna mengubah jumlah pesanan, memilih kategori, atau memperbarui keranjang, komponen Livewire dapat mengirim pembaruan ke server dan memperbarui bagian antarmuka yang relevan.

Pendekatan ini tidak berarti bahwa seluruh halaman menjadi SPA. Interaktivitas ditempatkan pada komponen yang benar-benar membutuhkannya.




*Gambar 1.1. Evolusi pendekatan rendering aplikasi web: SSR klasik, SPA, dan SSR dengan interaktivitas komponen.*

*Sumber: diadaptasi penulis (ilustrasi konseptual).*

Tabel 1.1 membandingkan ketiga pendekatan tersebut. Perbandingan ini menunjukkan kecenderungan umum, bukan aturan mutlak.

| Dimensi | SSR klasik | SPA | SSR + interaktivitas komponen |
|---|---|---|---|
| **Lokasi render awal** | Server | Umumnya klien | Server |
| **Pembaruan antarmuka** | Memuat ulang halaman atau sebagian halaman | Dikelola JavaScript di klien | Komponen tertentu diperbarui secara reaktif |
| **Kebutuhan JavaScript** | Rendah | Umumnya lebih tinggi | Sedang, bergantung jumlah komponen |
| **Kompleksitas state di klien** | Rendah | Cenderung tinggi | Sedang |
| **Kemudahan pengembangan** | Relatif sederhana | Bergantung pada pengalaman tim | Relatif sederhana bagi tim Laravel |
| **Kesesuaian umum** | Konten, formulir, aplikasi berbasis halaman | Antarmuka sangat interaktif | Aplikasi bisnis dengan kebutuhan interaksi terarah |

*Tabel 1.1. Perbandingan pendekatan rendering aplikasi web (data ilustratif).*

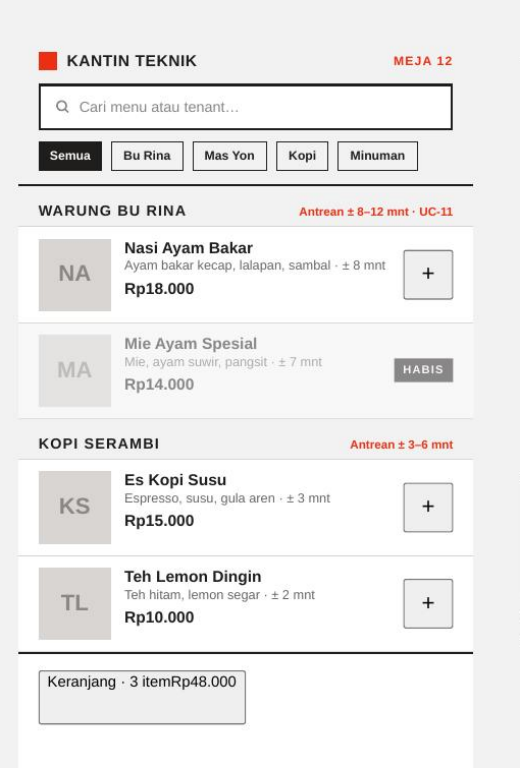
Tabel 1.1 memperlihatkan kecenderungan umum, bukan rekomendasi untuk satu proyek tertentu. Karena itu, keputusan rendering sebaiknya diambil setelah kriteria proyek dinyatakan lebih dahulu. Untuk studi kasus Kantin Multi-Tenant, kriteria tersebut adalah sebagai berikut.

1. Sebagian besar halaman berisi konten dan formulir: katalog menu, pengelolaan tenant, dan laporan.
2. Interaksi yang benar-benar reaktif hanya muncul pada sebagian kecil antarmuka, misalnya keranjang, filter menu, dan status pesanan.
3. Pengembang adalah satu tim kecil yang sudah terbiasa dengan alur kerja Laravel di sisi server.
4. Aplikasi tidak menyediakan antarmuka publik berbasis API untuk klien pihak ketiga pada tahap ini.
5. Waktu pengembangan terbatas pada satu semester, sehingga biaya belajar kerangka kerja klien perlu ditekan.

Berdasarkan kelima kriteria tersebut, kolom SSR dengan interaktivitas komponen pada Tabel 1.1 paling banyak memenuhi kebutuhan proyek. Keputusan ini bersifat kontekstual: proyek dengan antarmuka yang jauh lebih dinamis atau dengan kebutuhan klien seluler mandiri dapat sampai pada kesimpulan yang berbeda.

Untuk proyek Kantin Multi-Tenant, Blade dan Livewire menjadi pilihan yang masuk akal apabila tim ingin mempertahankan alur kerja Laravel berbasis server sekaligus menyediakan interaksi seperti pembaruan keranjang, filter menu, atau status pesanan. Pilihan ini selaras dengan berkas konfigurasi proyek, yang mengaktifkan Livewire dan Reverb untuk kebutuhan komponen reaktif dan pembaruan waktu nyata.

### 1.2.4 Membaca contoh antarmuka




*Gambar 1.2. Antarmuka pelanggan (katalog per tenant) aplikasi Kantin Multi-Tenant.*

*Sumber: berkas Mockup UI Kantin Multi-Tenant v2 (ilustratif).*

Pada antarmuka katalog, pengguna membutuhkan informasi yang mudah dipindai: nama menu, harga, ketersediaan stok, dan tindakan untuk menambahkan pesanan. Informasi tersebut dapat dirender pada halaman awal oleh server.

Komponen interaktif dapat digunakan pada bagian yang berubah, misalnya:

- Filter menu berdasarkan kategori.
- Pencarian menu.
- Jumlah item dalam keranjang.
- Status ketersediaan menu.
- Pembaruan status pesanan.

Pemilihan komponen harus proporsional. Jangan menambahkan reaktivitas hanya karena teknologi tersedia. Gunakan komponen interaktif ketika pembaruan sebagian halaman benar-benar meningkatkan pengalaman pengguna atau mengurangi pemuatan ulang yang tidak perlu.

## 1.3 Memilih Gaya Arsitektur Aplikasi

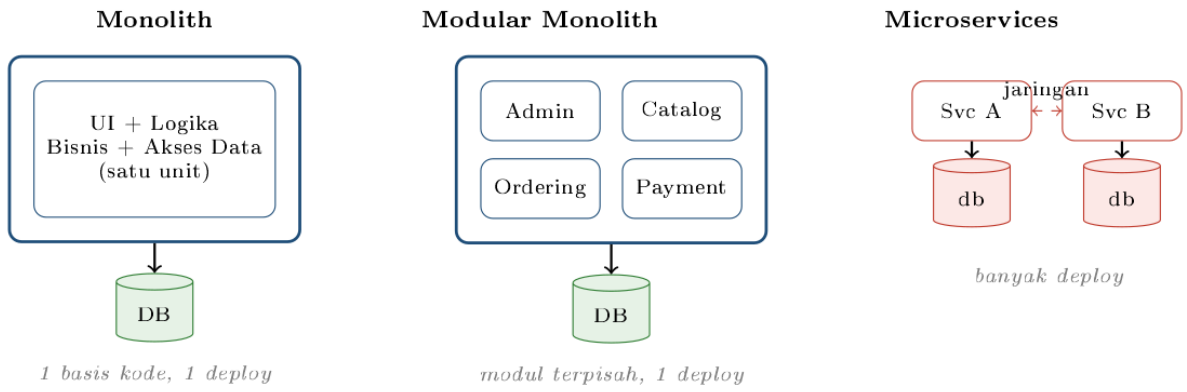
### 1.3.1 Monolith, modular monolith, dan microservices

Setelah memilih cara halaman dirender, keputusan berikutnya adalah bagaimana kode dan layanan aplikasi diorganisasikan. Tiga istilah yang sering digunakan adalah monolith, modular monolith, dan microservices.

**Monolith** adalah aplikasi yang dibangun dan diterapkan sebagai satu unit. Kode antarmuka, aturan bisnis, dan akses data dapat berada dalam satu repositori dan satu proses aplikasi. Monolith tidak selalu buruk atau tidak terstruktur; kualitasnya bergantung pada bagaimana kode dipisahkan dan dirawat.

**Modular monolith** tetap diterapkan sebagai satu unit, tetapi kode dibagi ke dalam modul dengan tanggung jawab dan batas yang jelas. Misalnya, aplikasi Kantin dapat memiliki modul Catalog, Ordering, Payments, Kitchen, Admin, dan Reporting.

**Microservices** memecah sistem menjadi beberapa layanan mandiri yang diterapkan secara terpisah. Setiap layanan biasanya berkomunikasi melalui jaringan, misalnya HTTP, gRPC, antrean pesan, atau peristiwa.




*Gambar 1.3. Perbandingan gaya arsitektur: monolith, modular monolith, dan microservices.*

*Sumber: diadaptasi penulis (ilustrasi konseptual).*

Tabel 1.2 merangkum karakter umum ketiga gaya tersebut.

| Kriteria | Monolith | Modular monolith | Microservices |
|---|---|---|---|
| **Unit penerapan** | Satu | Satu | Banyak |
| **Batas kode** | Dapat longgar atau terstruktur | Ditegaskan melalui modul | Ditegaskan melalui batas layanan |
| **Komunikasi utama** | Pemanggilan dalam proses | Pemanggilan antarmodul atau peristiwa internal | Komunikasi jaringan |
| **Kompleksitas operasional** | Rendah | Rendah hingga sedang | Tinggi |
| **Skalabilitas per bagian** | Terbatas pada unit aplikasi | Terbatas pada unit aplikasi | Dapat lebih selektif |
| **Kebutuhan observabilitas** | Dasar | Dasar hingga sedang | Tinggi |

| | | | |
|---|---|---|---|
| **Kesesuaian umum** | Produk awal atau aplikasi sederhana | Domain cukup jelas, tim kecil-menengah | Sistem besar, banyak tim, kebutuhan deploy mandiri |

*Tabel 1.2. Kriteria umum pemilihan gaya arsitektur (data ilustratif).*

### 1.3.2 Kriteria pemilihan

Anda tidak perlu memilih microservices hanya karena aplikasi memiliki banyak fitur. Banyak aplikasi menjadi lebih sulit dikelola setelah dipecah terlalu dini. Modularitas kode adalah kualitas desain tersendiri, bukan konsekuensi otomatis dari pola penerapan.

Gunakan pertanyaan berikut untuk membantu memilih arsitektur:

1. Apakah domain aplikasi sudah cukup jelas untuk dipisahkan menjadi modul atau layanan?
2. Apakah beberapa bagian aplikasi benar-benar harus dirilis secara mandiri?
3. Apakah tim memiliki kemampuan untuk mengelola komunikasi jaringan, pemantauan, keamanan layanan, kegagalan parsial, dan pelacakan log lintas layanan?
4. Apakah terdapat bagian aplikasi dengan kebutuhan skala yang sangat berbeda?
5. Apakah manfaat pemisahan layanan lebih besar daripada biaya operasionalnya?

Untuk proyek pembelajaran atau proyek dengan satu tim kecil dan target rilis tunggal, modular monolith sering menjadi pilihan yang lebih seimbang. Tim memperoleh batas modul yang jelas tanpa harus langsung menangani kompleksitas sistem terdistribusi.

### 1.3.3 Amdahl's Law sebagai batas percepatan

Hukum Amdahl membantu kita memahami batas percepatan ketika sebagian pekerjaan dapat dijalankan secara paralel.

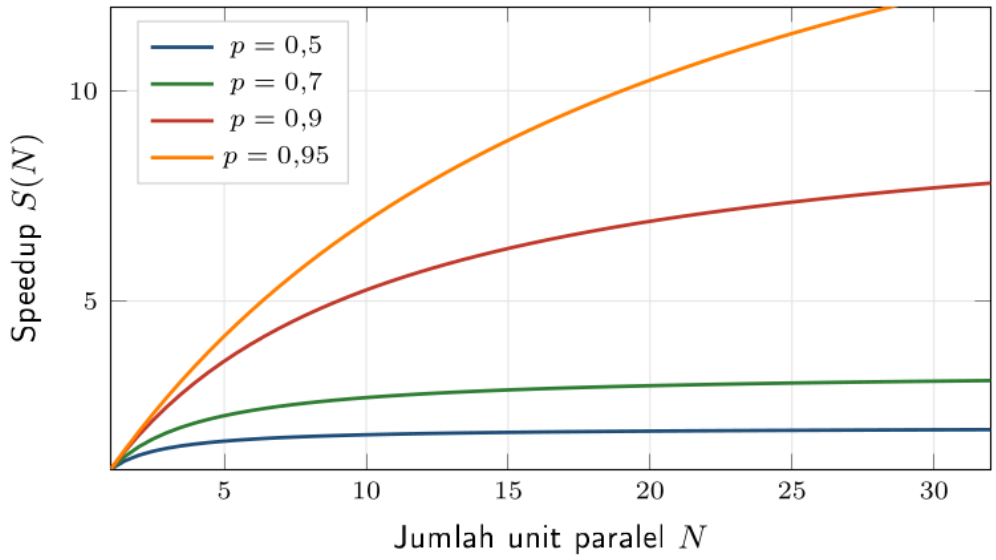
$$S(N) = \frac{1}{(1 - p) + \frac{p}{N}} \tag{1.1}$$

*Bentuk teks:* `S(N) = 1 / ( (1 - p) + p / N )`

dengan *S(N)* percepatan total ketika menggunakan *N* unit paralel, *p* bagian pekerjaan yang dapat diparalelkan (0–1), dan *N* jumlah unit paralel (mis. proses pekerja atau replika layanan).

Setiap persamaan pada bab ini disajikan sebagai gambar sekaligus dalam bentuk teks. Bentuk teks dapat Anda salin ke lembar kerja, kalkulator, atau kode program ketika mengerjakan latihan.

**Contoh 1.1. Percepatan dengan delapan unit paralel.** Misalkan 90 persen pekerjaan dapat diparalelkan, sehingga *p* = 0,9. Jika tersedia delapan unit paralel, maka S(8) = 1 / (0,1 + 0,9/8) = 1 / 0,2125 ≈ 4,7. Artinya, secara teoretis pekerjaan dapat menjadi sekitar 4,7 kali lebih cepat dibandingkan kondisi satu unit. Namun, percepatan maksimum ketika *N* mendekati tak hingga adalah 1 / (1 − 0,9) = 10. Bagian pekerjaan yang tidak dapat diparalelkan tetap menjadi penghambat utama.




*Gambar 1.4. Kurva percepatan Amdahl untuk beberapa fraksi paralel p (data ilustratif).*

*Sumber: perhitungan penulis dengan pgfplots (data ilustratif).*

**Pembacaan contoh.** Menambah unit pemrosesan tidak selalu menghasilkan peningkatan yang sebanding. Jika basis data, proses pembayaran, atau aturan transaksi masih menjadi bagian serial, penambahan layanan atau server tidak otomatis menyelesaikan masalah kinerja.

**Peringatan.** Hukum Amdahl bukan rumus untuk memutuskan bahwa aplikasi harus dipecah menjadi microservices. Rumus ini hanya membantu menganalisis batas percepatan akibat paralelisasi. Keputusan arsitektur tetap memerlukan pertimbangan domain, organisasi tim, data, keamanan, dan biaya operasional.

### 1.3.4 Little's Law untuk memperkirakan konkurensi

Hukum Little menghubungkan jumlah rata-rata permintaan yang berada dalam sistem dengan laju kedatangan dan waktu penyelesaian rata-rata.

$$L = \lambda \cdot W \tag{1.2}$$

*Bentuk teks:* `L = lambda * W`

dengan *L* jumlah rata-rata permintaan dalam sistem, *λ* laju kedatangan permintaan per detik, dan *W* waktu rata-rata sebuah permintaan berada dalam sistem (detik).

**Contoh 1.2. Estimasi permintaan bersamaan.** Jika aplikasi menerima rata-rata 20 permintaan per detik dan waktu penyelesaian rata-rata adalah 0,15 detik, maka L = 20 × 0,15 = 3. Secara rata-rata, terdapat tiga permintaan dalam sistem pada waktu yang sama.

**Pembacaan contoh.** Nilai tersebut membantu Anda memahami tingkat konkurensi rata-rata. Nilai itu belum cukup untuk menentukan jumlah *worker*, koneksi basis data, atau kapasitas produksi secara final. Anda tetap perlu mempertimbangkan lonjakan trafik, jenis permintaan, batas koneksi, penggunaan memori, serta hasil pengujian beban.

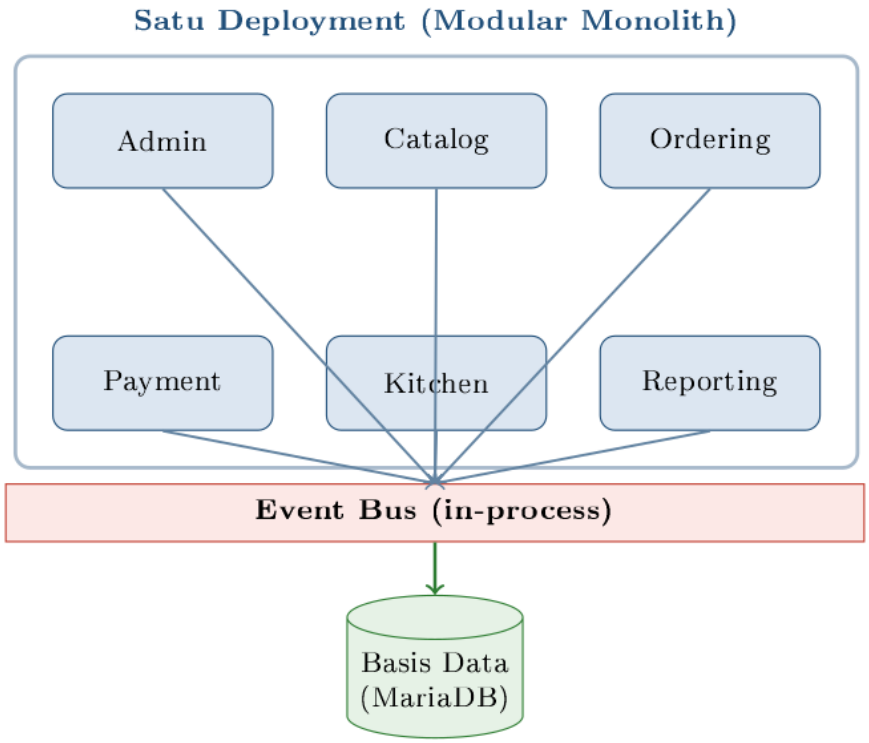
**Catatan.** Hukum Little berlaku pada sistem yang stabil, yaitu ketika laju kedatangan rata-rata sama dengan laju penyelesaian rata-rata selama periode pengamatan, dan ketika satuan *λ* serta

*W* konsisten (misalnya permintaan per detik dan detik). Hasilnya adalah estimasi rata-rata konkurensi pada periode tersebut, bukan konfigurasi akhir jumlah worker atau koneksi basis data. Pada sistem yang sedang jenuh atau pada periode lonjakan, antrean bertambah panjang dan estimasi ini tidak lagi mewakili keadaan sesungguhnya.

### 1.3.5 Penerapan pada proyek Kantin

Dalam studi kasus ini, aplikasi Kantin Multi-Tenant dirancang sebagai modular monolith. Aplikasi diterapkan sebagai satu unit, tetapi kode dikelompokkan berdasarkan tanggung jawab bisnis. Contoh pembagian modul adalah:

- Admin untuk pengelolaan pengguna, tenant, dan pengaturan.
- Catalog untuk pengelolaan menu dan kategori.
- Ordering untuk keranjang, pesanan, dan status pesanan.
- Payments untuk proses atau pencatatan pembayaran.
- Kitchen untuk alur produksi dan penyelesaian pesanan.
- Reporting untuk ringkasan dan laporan.




*Gambar 1.5. Peta modul modular monolith aplikasi Kantin Multi-Tenant.*

*Sumber: diadaptasi dari SRS Aplikasi Kantin, Lampiran B (ilustratif).*

Pembagian tersebut membantu tim menentukan lokasi kode, tanggung jawab data, dan aturan komunikasi antarmodul. Pada proyek, setiap modul berkomunikasi melalui kontrak dan peristiwa, bukan mengambil langsung detail internal modul lain. Jika pada masa mendatang salah satu modul benar-benar membutuhkan skala atau siklus rilis yang berbeda, modul tersebut dapat dievaluasi kembali untuk dipisahkan. Namun, pemisahan itu sebaiknya dilakukan berdasarkan bukti kebutuhan, bukan asumsi.

## 1.4 Framework Laravel dan Struktur Aplikasi

### 1.4.1 Mengapa menggunakan framework

*Web framework* adalah kumpulan pustaka, konvensi, dan perkakas yang membantu pengembang membangun aplikasi web secara terstruktur. Framework mengurangi pekerjaan berulang, misalnya penanganan rute, validasi input, autentikasi, migrasi basis data, dan pengujian.

Laravel digunakan dalam studi kasus ini karena menyediakan komponen untuk membangun aplikasi PHP modern, antara lain Blade untuk tampilan, Eloquent untuk akses data, Artisan untuk perintah terminal, serta *service container* untuk pengelolaan dependensi. Pada proyek, dependensi utama mencakup *framework* Laravel 13, Livewire 4, dan Reverb, dengan autentikasi ditangani oleh Laravel Fortify.

Tabel 1.3 menunjukkan beberapa contoh framework web. Daftar ini bersifat pengantar dan tidak dimaksudkan sebagai peringkat.

| Framework | Bahasa | Ciri umum |
|---|---|---|
| **Laravel** | PHP | Ekosistem aplikasi web; Blade, Eloquent, Artisan |
| **Django** | Python | ORM, panel admin, pendekatan *batteries included* |
| **Spring Boot** | Java | Dukungan aplikasi *enterprise* dan *dependency injection* |
| **NestJS** | TypeScript | Struktur modular untuk aplikasi sisi server |
| **Ruby on Rails** | Ruby | Konvensi kuat dan produktivitas pengembangan |

*Tabel 1.3. Contoh framework web populer (data ilustratif).*

Framework mempercepat pekerjaan, tetapi tidak menggantikan pemahaman dasar. Anda tetap perlu memahami alur permintaan, validasi, keamanan, struktur data, dan pengujian.

### 1.4.2 MVC sebagai pemisahan tanggung jawab

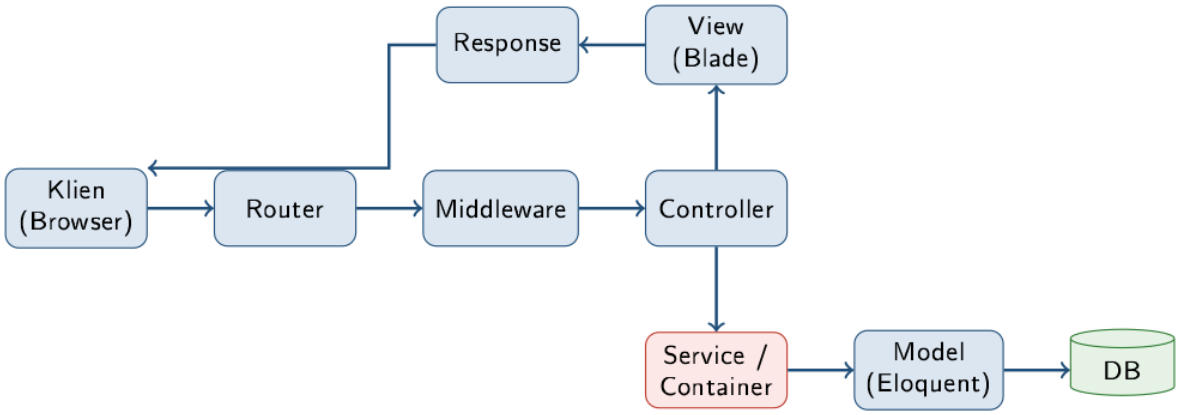
MVC adalah singkatan dari *Model–View–Controller*. Pola ini membagi aplikasi ke dalam tiga peran utama.

- **Model** menangani data dan aturan yang terkait dengan data.
- **View** menyajikan data kepada pengguna.
- **Controller** menerima permintaan, mengarahkan proses, dan menentukan respons.

Sebagai contoh, ketika pelanggan membuka daftar menu:

1. Rute menerima permintaan ke halaman menu.
2. Controller meminta data kepada model atau *service*.
3. Model mengambil data yang diperlukan dari basis data.
4. Controller mengirim data ke *view*.
5. View membentuk HTML untuk dikirim kepada pengguna.

Pembagian tersebut tidak berarti setiap aturan bisnis harus diletakkan di controller. Jika controller menjadi terlalu besar, aturan bisnis dapat dipindahkan ke kelas *service* atau kelas domain yang lebih terfokus. Pada proyek Kantin, logika seperti perhitungan keranjang dan checkout ditempatkan pada kelas *service* di dalam modul Ordering, bukan di dalam controller.




*Gambar 1.6. Alur permintaan MVC dan service container pada Laravel.*

*Sumber: diadaptasi penulis (ilustrasi konseptual).*

### 1.4.3 Service container dan dependency injection

*Service container* adalah mekanisme Laravel untuk membuat dan menyediakan objek yang dibutuhkan kelas lain. Konsep ini berkaitan dengan *dependency injection*, yaitu pemberian dependensi dari luar kelas, bukan membuat seluruh dependensi secara langsung di dalam kelas.

Misalnya, sebuah controller memerlukan sebuah *service*. Daripada controller membuat objek tersebut sendiri, Laravel dapat menyuntikkannya melalui *constructor*. Pendekatan ini membuat kode lebih mudah diuji karena dependensi dapat diganti dengan implementasi lain atau objek tiruan saat pengujian.

**Contoh dependency injection melalui constructor (ilustratif)**

```php
// Sebelum: controller membuat sendiri dependensinya.
final class MenuController extends Controller
{
    public function index()
    {
        $catalog = new CatalogService();   // sulit diganti saat pengujian
        return view('menu.index', ['menus' => $catalog->menuTersedia()]);
    }
}

// Sesudah: dependensi diminta melalui constructor.
final class MenuController extends Controller
{
    public function __construct(
        private readonly CatalogService $catalog,
    ) {
    }

    public function index()
    {
        return view('menu.index', ['menus' => $this->catalog->menuTersedia()]);
    }
}
```

Pada bentuk kedua, controller tidak lagi menentukan cara pembuatan objek. Container yang menyediakannya, sehingga pada pengujian `CatalogService` dapat digantikan objek tiruan tanpa mengubah kode controller.

Pada aplikasi Kantin yang berbentuk modular monolith, setiap modul memiliki satu *service provider* sebagai titik perakitan: pendaftaran *binding* pada metode `register()`, serta rute, peristiwa, atau

*policy* pada metode `boot()`. Kode 1.1 menunjukkan kerangka *service provider* modul Catalog pada proyek.

**app/Modules/Catalog/CatalogServiceProvider.php**

```php
<?php

namespace App\Modules\Catalog;

use Illuminate\Support\ServiceProvider;

final class CatalogServiceProvider extends ServiceProvider
{
    public function register(): void
    {
        // Binding kontrak -> implementasi ditambahkan
        // saat modul Catalog diimplementasikan (Modul 7).
    }

    public function boot(): void
    {
        // Rute, event listener, dan policy modul didaftarkan di sini.
    }
}
```

**Kode 1.1.** Kerangka *service provider* modul Catalog. Kelas berada pada *namespace* `App\Modules\Catalog` dan ditandai `final` agar batas modul tetap tegas.

Berbeda dengan pola lama yang memuat rute langsung di dalam provider, sejak Laravel 12 setiap provider modul didaftarkan pada berkas `bootstrap/providers.php`. Berkas inilah yang menegakkan batas satu provider per *bounded context*.

**bootstrap/providers.php**

```php
<?php

use App\Modules\Admin\AdminServiceProvider;
use App\Modules\Catalog\CatalogServiceProvider;
use App\Modules\Ordering\OrderingServiceProvider;
use App\Modules\Payments\PaymentsServiceProvider;
use App\Modules\Kitchen\KitchenServiceProvider;
use App\Modules\Reporting\ReportingServiceProvider;
use App\Providers\AppServiceProvider;

return [
    AppServiceProvider::class,
    // Modular monolith: satu provider per bounded context.
    AdminServiceProvider::class,
    CatalogServiceProvider::class,
    OrderingServiceProvider::class,
    PaymentsServiceProvider::class,
    KitchenServiceProvider::class,
    ReportingServiceProvider::class,
];
```

Ketika mendaftarkan dependensi, Anda memilih *lifecycle*-nya. Metode `bind` membuat instance baru setiap kali kelas diselesaikan. Metode `singleton` membuat satu instance yang dipakai ulang selama siklus hidup container aplikasi. Metode `scoped` membuat satu instance per *request* atau *job*, lalu mereset instance tersebut pada siklus berikutnya. Pilihan bergantung pada kebutuhan objek, bukan kebiasaan.

Sebagai contoh nyata, `TenantContext` pada proyek Kantin didaftarkan sebagai `scoped`, karena konteks tenant harus berlaku satu instance per *request* dan tidak boleh bocor ke permintaan lain.

```
app/Providers/AppServiceProvider.php  (kutipan register())
public function register(): void
{
    // Scoped: satu instance per request/job;
    // direset pada lifecycle berikutnya (tidak bocor antarpermintaan).
    $this->app->scoped(
        TenantContext::class,
        fn (): TenantContext => new TenantContext,
    );
}
```

**Penjelasan input–proses–output.** Input: kelas yang perlu disediakan container. Proses: `register()` mengikat kelas ke container, `boot()` memuat rute dan berkas pendukung modul. Output: modul siap dipakai controller lain melalui *dependency injection*, tanpa mencampur kode antarmodul. Interpretasi: pola ini menjaga batas modular monolith sekaligus memudahkan pengujian karena dependensi dapat digantikan tiruan.

> **Perlu verifikasi.** Pola *binding* yang paling tepat untuk sebuah layanan (`bind`, `singleton`, atau `scoped`) sebaiknya selalu disesuaikan dengan kebutuhan siklus hidup objek pada versi Laravel yang digunakan proyek.

## 1.5 Menyiapkan Lingkungan Pengembangan

### 1.5.1 Prinsip lingkungan yang dapat diulang

Lingkungan pengembangan yang baik memungkinkan anggota tim memasang perangkat dan dependensi yang konsisten. Prinsip ini disebut *reproducibility*: hasil pemasangan tidak bergantung pada konfigurasi pribadi satu pengembang.

Komponen utama yang digunakan dalam studi kasus ini dirangkum pada Tabel 1.4.

| Komponen | Peran |
|---|---|
| **Composer** | Mengelola paket PHP dan *autoload* |
| **NPM dan Vite** | Mengelola serta membangun aset *front-end* |
| **MariaDB** | Menyimpan data relasional dan transaksi |
| **Redis** | Menyediakan *cache*, sesi, dan antrean |
| **Reverb** | Server WebSocket untuk fitur *real-time* |
| **Git** | Merekam riwayat serta mendukung kolaborasi kode |
| **Pint, Larastan, PHPUnit** | Format kode, analisis statis, dan pengujian |

*Tabel 1.4. Komponen lingkungan pengembangan dan perannya.*

Tabel 1.5 menampilkan versi acuan yang digunakan proyek. Nilai versi bahasa dan *framework* mengikuti berkas `composer.json`, sedangkan versi layanan (MariaDB, Redis, Node.js) mengikuti lingkungan pengembangan dan sebaiknya dipastikan kembali sebelum praktikum dimulai.

| Parameter | Nilai acuan |
|---|---|
| **PHP** | 8.3+ |

| | |
|---|---|
| **Laravel** | 13.x |
| **Livewire** | 4.x |
| **Reverb** | 1.x |
| **MariaDB** | 10.6+ (diakses via driver *mysql* Laravel) |
| **Redis** | 7.x |
| **Node.js** | LTS (mis. 20 atau 22) |

*Tabel 1.5. Parameter versi lingkungan acuan.*

**Peringatan.** Seluruh nilai pada Tabel 1.5 adalah konfigurasi acuan, bukan jaminan kompatibilitas. Sebelum praktikum dimulai, pastikan kembali kecocokan versi PHP, Laravel, Livewire, Reverb, MariaDB, Redis, Node.js, serta paket quality gate terhadap berkas `composer.json`, `package.json`, dan dokumentasi resmi masing-masing perangkat. Ketidakcocokan versi adalah penyebab kegagalan penyiapan lingkungan yang paling sering terjadi.

### 1.5.2 Urutan penyiapan awal

Gunakan urutan berikut agar kesalahan lebih mudah dilacak.

1. Pastikan PHP, Composer, Node.js, NPM, MariaDB, Redis, dan Git sudah tersedia.
2. Periksa versi perangkat yang digunakan.
3. Ambil kode proyek dan pasang dependensi PHP serta JavaScript.
4. Salin dan sesuaikan berkas konfigurasi lingkungan (`.env`).
5. Buat basis data lokal.
6. Jalankan migrasi basis data.
7. Jalankan aplikasi dan bangun aset *front-end*.
8. Jalankan pemeriksaan kualitas awal.

Kode 1.2 memperlihatkan langkah pembuatan proyek dan pemasangan dependensi.

```
Terminal
# 1. Ambil proyek dan masuk ke direktorinya
git clone <url-repositori-proyek> kantin-multi-tenant
cd kantin-multi-tenant

# 2. Pasang dependensi PHP dan JavaScript
composer install
npm install

# 3. Siapkan berkas konfigurasi dan kunci aplikasi
cp .env.example .env
php artisan key:generate
```

**Kode 1.2.** Pengambilan kode proyek dan pemasangan dependensi. Tiga langkah berurutan: mengambil repositori, memasang dependensi PHP serta JavaScript, lalu menyiapkan berkas `.env` dan kunci aplikasi.

Setelah itu, buat basis data lokal sesuai nama yang akan dimasukkan ke berkas `.env`. Pada sistem operasi tertentu, perintah untuk membuat basis data dapat berbeda. Anda dapat menggunakan klien MariaDB, alat administrasi basis data, atau perintah SQL yang sesuai dengan lingkungan lokal. Kemudian jalankan migrasi dan server pengembangan.

```
Terminal
php artisan migrate
npm run dev
php artisan serve
```

Jika berhasil, aplikasi pengembangan biasanya dapat diakses melalui alamat yang ditampilkan oleh Artisan. Jangan menganggap nomor port selalu sama; periksa keluaran terminal Anda.

> **Tip.** Proyek menyediakan pintasan Composer untuk menghemat langkah: `composer setup` menjalankan pemasangan, penyalinan `.env`, pembuatan kunci, migrasi, dan pembangunan aset sekaligus, sedangkan `composer dev` menjalankan server pengembangan.

### 1.5.3 Manajemen dependensi dan lockfile

Composer mengelola dependensi PHP, sedangkan NPM mengelola dependensi JavaScript. Berkas seperti `composer.lock` dan *lockfile* NPM menyimpan versi dependensi yang dipasang secara spesifik.

Berkas *lock* penting karena dua pengembang dapat memperoleh hasil berbeda apabila mereka hanya mengandalkan rentang versi pada berkas konfigurasi utama.

Dalam *semantic versioning*, versi umumnya ditulis sebagai MAYOR.MINOR.PATCH.

- Versi mayor menunjukkan perubahan yang berpotensi tidak kompatibel.
- Versi minor menunjukkan penambahan fitur yang kompatibel.
- Versi patch menunjukkan perbaikan masalah yang kompatibel.

Sebagai contoh, batasan `^13.17` pada berkas `composer.json` proyek mengizinkan pembaruan minor dan patch dalam batas versi mayor yang sama. Detail perilaku batas versi tetap harus diperiksa melalui dokumentasi Composer.

## 1.6 Basis Data, Cache, dan Quality Gate

### 1.6.1 MariaDB dan Redis

MariaDB adalah sistem manajemen basis data relasional yang menyimpan data secara persisten. Dalam aplikasi Kantin, data seperti pengguna, tenant, menu, pesanan, dan pembayaran biasanya memerlukan penyimpanan transaksional. Pada berkas konfigurasi, MariaDB diakses melalui koneksi `mysql` bawaan Laravel yang kompatibel dengannya.

Redis adalah penyimpanan data di memori yang dapat digunakan untuk *cache*, sesi, antrean, atau kebutuhan lain sesuai konfigurasi aplikasi. Cache membantu mengurangi pembacaan berulang dari basis data, terutama untuk data yang sering dibaca tetapi tidak sering berubah.

Misalnya, daftar menu tenant dapat disimpan sementara dalam *cache*. Ketika pelanggan membuka katalog, aplikasi dapat menggunakan data *cache* apabila data tersebut masih valid. Jika menu berubah, cache harus diperbarui atau dihapus agar pengguna tidak menerima informasi lama.

### 1.6.2 Menghitung latensi cache secara ilustratif

Latensi efektif rata-rata dapat diperkirakan dengan persamaan berikut.

$$L_{\text{eff}} = h \cdot L_{\text{cache}} + (1 - h) \cdot L_{\text{db}} \tag{1.3}$$

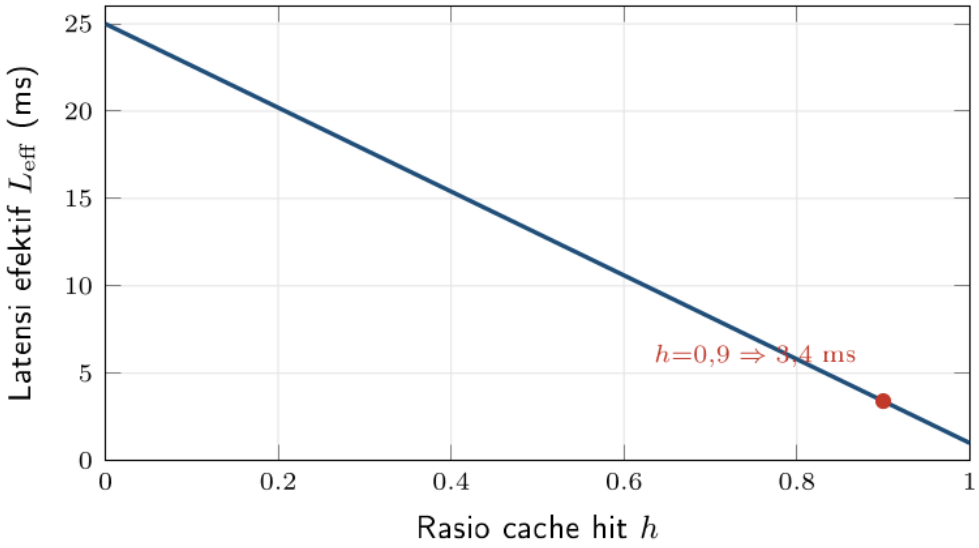
*Bentuk teks:* `L_eff = h * L_cache + (1 - h) * L_db`

dengan *L*_eff latensi efektif rata-rata, *h* rasio *cache hit* (0–1), *L*_cache latensi akses cache, dan *L*_db latensi akses basis data.

**Contoh 1.3. Dampak rasio cache hit.** Misalkan latensi cache adalah 1 ms dan latensi basis data adalah 25 ms. Jika rasio *cache hit* sebesar 0,9, maka L_eff = 0,9(1) + 0,1(25) = 3,4 ms. Hasil tersebut lebih rendah daripada 25 ms, yaitu latensi ketika semua permintaan langsung menuju basis data.

| Rasio cache hit | Latensi efektif | Perbandingan terhadap tanpa cache |
|---|---|---|
| **0,0** | 25,0 ms | 1,0× |
| **0,5** | 13,0 ms | 1,9× |
| **0,8** | 5,8 ms | 4,3× |
| **0,9** | 3,4 ms | 7,4× |
| **0,95** | 2,2 ms | 11,4× |

*Tabel 1.6. Latensi efektif berdasarkan rasio cache hit (data ilustratif).*




*Gambar 1.7. Latensi efektif sebagai fungsi rasio cache hit (data ilustratif).*

*Sumber: perhitungan penulis dengan pgfplots (data ilustratif).*

**Pembacaan contoh.** Cache memberikan manfaat terbesar ketika data sering dibaca dan rasio *cache hit* tinggi. Namun, cache juga menambah persoalan konsistensi. Data menu yang sudah tidak tersedia tidak boleh tetap muncul sebagai stok tersedia hanya karena cache belum diperbarui.

> **Peringatan.** Cache bukan pengganti basis data dan bukan sumber kebenaran utama untuk data transaksi. Untuk data yang memengaruhi pembayaran, jumlah stok kritis, atau status pesanan, strategi konsistensi harus dirancang dengan hati-hati.

### 1.6.3 Contoh konfigurasi lokal

Kode 1.3 berikut adalah kutipan konfigurasi lokal berdasarkan berkas `.env.example` proyek. Nilai nama basis data, pengguna, kata sandi, dan *driver* harus disesuaikan dengan lingkungan Anda.

```
.env  (kutipan)
```

```
APP_NAME="Kantin Multi-Tenant"
APP_ENV=local
APP_LOCALE=id

DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=kantin
DB_USERNAME=kantin
DB_PASSWORD=GANTI_DENGAN_KATA_SANDI_LOKAL

SESSION_DRIVER=redis
CACHE_STORE=redis
QUEUE_CONNECTION=redis
BROADCAST_CONNECTION=reverb

REDIS_CLIENT=phpredis
REDIS_HOST=127.0.0.1
REDIS_PASSWORD=null
REDIS_PORT=6379
```

**Kode 1.3.** Kutipan berkas `.env` untuk lingkungan lokal. Seluruh nilai rahasia ditulis sebagai penanda dan wajib diganti pada komputer masing-masing.

Nilai `DB_CONNECTION=mysql` adalah nama koneksi bawaan Laravel yang tetap kompatibel dengan server MariaDB. Nilai `DB_PASSWORD` di sini hanya penanda; ganti dengan kata sandi lokal Anda.

> **Peringatan.** Jangan menyimpan kata sandi, token, atau kredensial produksi di repositori Git. Berkas `.env` umumnya bersifat lokal dan sudah tercantum dalam `.gitignore` proyek. Gunakan mekanisme pengelolaan rahasia yang sesuai ketika aplikasi diterapkan ke lingkungan produksi.

### 1.6.4 Pemeriksaan koneksi Redis

Sebelum menggunakan Redis untuk cache atau antrean, Anda dapat melakukan pemeriksaan sederhana terhadap koneksi. Kode 1.4 bersifat ilustratif untuk pemeriksaan lokal: kunci uji diberi awalan khusus dan masa berlaku singkat agar tidak meninggalkan data cache yang tidak perlu.

**Contoh perintah Artisan (kutipan handle())**

```php
<?php

use Illuminate\Support\Facades\Redis;

public function handle(): int
{
    try {
        $key = 'healthcheck:redis';
        Redis::setex($key, 30, 'ok');       // TTL 30 detik

        $value = Redis::get($key);
        if ($value !== 'ok') {
            $this->error('Redis terhubung, tetapi nilai uji tidak sesuai.');

            return self::FAILURE;
        }

        $this->info('Redis sehat: baca dan tulis berhasil.');

        return self::SUCCESS;
    } catch (\Throwable $exception) {
        $this->error('Gagal terhubung ke Redis: ' . $exception->getMessage());
```

```
        return self::FAILURE;
    }
}
```

**Kode 1.4.** Pemeriksaan koneksi Redis. Kunci uji diberi awalan `healthcheck:` dan masa berlaku 30 detik agar tidak mengganggu data cache aplikasi.

Kode tersebut menulis nilai ok, membaca kembali nilainya, lalu mengembalikan status berhasil atau gagal. Masa berlaku 30 detik mencegah kunci uji tersimpan terlalu lama.

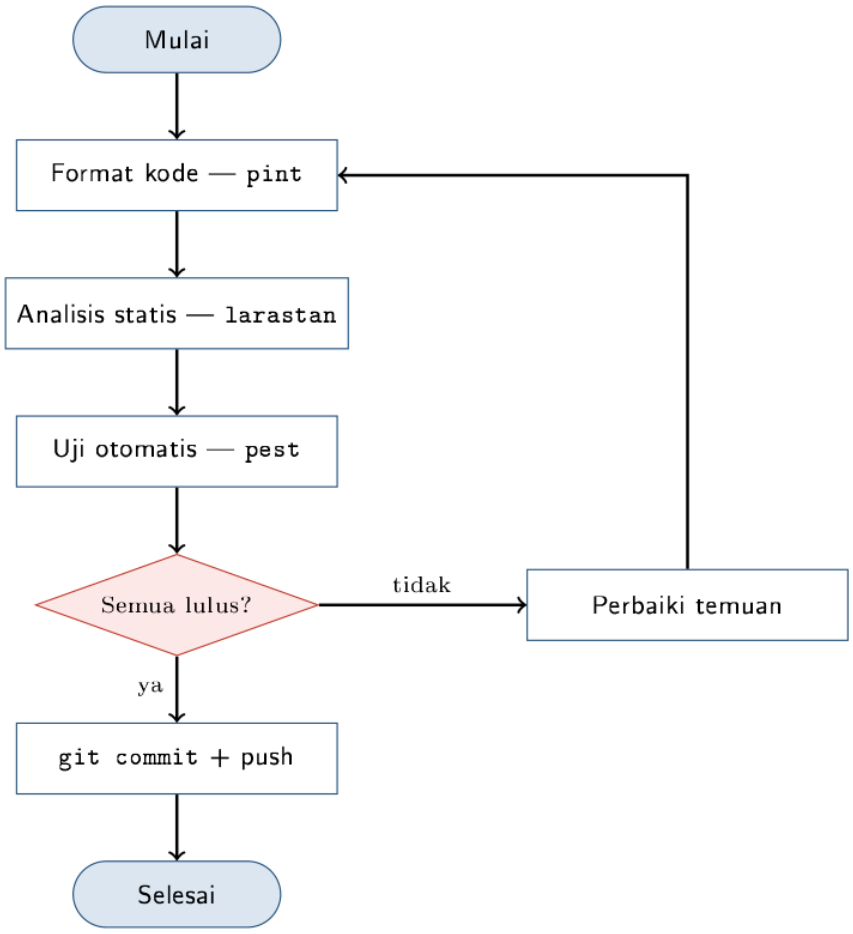
> **Catatan.** Selain pemeriksaan manual seperti ini, aplikasi Laravel proyek sudah menyediakan *endpoint* kesehatan bawaan pada alamat `/up`. Endpoint tersebut berguna untuk pemantauan otomatis, sedangkan pemeriksaan Redis di atas berfokus pada satu layanan.

### 1.6.5 Git dan quality gate

Git adalah sistem kendali versi yang menyimpan riwayat perubahan kode. Git membantu tim mengetahui siapa yang mengubah suatu berkas, membandingkan perubahan, membuat *branch*, serta menggabungkan pekerjaan secara terkontrol.

*Quality gate* adalah serangkaian pemeriksaan yang harus dilalui kode sebelum digabungkan. Pada proyek ini, quality gate terdiri atas:

- **Pint** untuk memeriksa atau menerapkan format kode.
- **Larastan/PHPStan** untuk analisis statis.
- **PHPUnit** (melalui `php artisan test`) untuk menjalankan pengujian.



*Gambar 1.8. Alur quality gate lokal sebelum commit.*

*Sumber: diadaptasi penulis (ilustrasi konseptual).*

Kode 1.5 menunjukkan skrip *quality gate* yang tersedia pada berkas `composer.json` proyek.

`composer.json  (kutipan bagian scripts)`

```
{
    "scripts": {
        "lint:check": [
            "pint --parallel --test"
        ],
        "types:check": [
            "phpstan analyse"
        ],
        "test": [
            "@php artisan config:clear --ansi",
            "@lint:check",
            "@types:check",
            "@php artisan test"
        ]
    }
}
```

**Kode 1.5.** Skrip quality gate pada berkas `composer.json`. Skrip `test` merangkai pemeriksaan format, analisis statis, dan pengujian menjadi satu perintah.

Perintah `composer test` menjalankan pembersihan konfigurasi, pemeriksaan format (`pint --parallel --test`), analisis statis (`phpstan analyse`), lalu pengujian (`php artisan test`) secara berurutan. Jalankan pemeriksaan tersebut sebelum mengirim perubahan.

`Terminal`

```
# Menjalankan seluruh quality gate; berhenti bila ada tahap yang gagal
composer test

# Jika lulus, buat branch mingguan lalu commit dan push
git checkout -b feat/week-01-setup
git add -A
git commit -m "Modul 1: setup lingkungan dan quality gate"
git push origin feat/week-01-setup
```

**Kode 1.6.** Menjalankan quality gate lalu mengirim perubahan. Perintah `composer test` dijalankan lebih dahulu; commit dan push hanya dilakukan apabila seluruh tahap lulus.

**Kode 1.5** dan **Kode 1.6** memperlihatkan skrip serta perintah *commit* tersebut. Jika satu pemeriksaan gagal, selesaikan masalahnya sebelum perubahan digabungkan. Pada tahap awal, Anda dapat menjalankan perintah ini secara manual. Dalam proyek yang lebih matang, quality gate yang sama juga dijalankan oleh *continuous integration* (CI) ketika *pull request* dibuat; berkas alur kerja CI proyek berada pada direktori `.github`.

Perlu dibedakan antara menjalankan pemeriksaan dan menegakkan pemeriksaan. Terdapat tiga tingkat penegakan yang lazim digunakan.

- Pemeriksaan manual lokal: pengembang menjalankan `composer test` sendiri sebelum commit. Paling sederhana, tetapi bergantung pada kedisiplinan masing-masing orang.
- Git hook: berkas hook seperti `pre-commit` atau `pre-push` menjalankan pemeriksaan secara otomatis di komputer pengembang. Hook memberi umpan balik cepat, tetapi dapat dilewati dengan opsi `--no-verify` dan harus dipasang ulang oleh setiap anggota tim.

- Continuous integration (CI): pemeriksaan dijalankan di server ketika pull request dibuat. Inilah satu-satunya tingkat yang benar-benar menegakkan aturan, karena hasilnya tidak bergantung pada konfigurasi komputer pengembang dan dapat dijadikan syarat penggabungan.

Contoh pada bab ini berada pada tingkat pertama. Tingkat kedua dan ketiga dibahas lebih lanjut ketika alur kerja tim diperkenalkan.

> **Tip.** Gunakan pesan *commit* yang menjelaskan perubahan utama. Pesan yang spesifik memudahkan tim menelusuri riwayat dan menghubungkan perubahan dengan tugas atau isu yang dikerjakan.

## 1.7 Studi Kasus Terintegrasi: Kantin Multi-Tenant

Tim pengembang Kantin Teknik akan membangun aplikasi untuk mengelola tenant, katalog menu, pesanan, pembayaran, proses dapur, dan laporan. Tim terdiri atas beberapa mahasiswa dalam satu proyek semester dengan target rilis tunggal.

**Langkah pertama** adalah memilih gaya arsitektur. Tim tidak langsung memilih microservices hanya karena aplikasi memiliki enam area fungsi. Mereka menilai bahwa aplikasi masih dikerjakan oleh satu tim, diterapkan sebagai satu produk, dan belum memerlukan rilis mandiri untuk setiap fungsi. Karena itu, tim memilih modular monolith. Setiap area fungsi dibentuk sebagai modul dengan batas tanggung jawab yang jelas. Sebagai contoh, modul Ordering dapat menggunakan data dari Catalog melalui kontrak atau layanan yang disepakati, bukan mengambil langsung detail internal modul lain.

**Langkah kedua** adalah menyiapkan lingkungan. Tim menyepakati versi perangkat, memasang dependensi melalui Composer dan NPM, membuat basis data lokal, mengatur `.env`, serta menjalankan migrasi. Kesepakatan ini mengurangi masalah “aplikasi hanya berjalan di komputer saya”.

**Langkah ketiga** adalah menegakkan kualitas sejak awal. Sebelum perubahan digabungkan, tim menjalankan Pint, Larastan, dan PHPUnit melalui `composer test`. Jika pemeriksaan gagal, perubahan diperbaiki terlebih dahulu.

**Analisis studi kasus.** Modular monolith tidak menjamin aplikasi langsung rapi. Kerapian tetap bergantung pada disiplin batas modul, struktur kode, pengujian, dan dokumentasi keputusan. Namun, untuk proyek dengan satu tim dan satu target rilis, pilihan tersebut dapat memberi keseimbangan antara struktur yang baik dan beban operasional yang masih realistis.

## 1.8 Rangkuman, Istilah, dan Latihan

### Rangkuman Bab

- Arsitektur aplikasi web menentukan pembagian tanggung jawab antara antarmuka, aturan bisnis, data, dan layanan pendukung.
- SSR membentuk halaman di server, SPA banyak mengelola antarmuka di klien, sedangkan SSR dengan interaktivitas komponen menggabungkan render awal di server dengan pembaruan bagian tertentu dari halaman.
- Monolith, modular monolith, dan microservices memiliki konsekuensi berbeda terhadap penerapan, komunikasi, skalabilitas, dan operasi sistem.

- Hukum Amdahl membantu memahami batas percepatan akibat paralelisasi, sedangkan Hukum Little membantu memperkirakan rata-rata jumlah permintaan dalam sistem.
- Laravel mendukung struktur aplikasi melalui MVC, *service container*, Artisan, dan ekosistem paket.
- Lingkungan pengembangan yang dapat diulang memerlukan manajemen dependensi, konfigurasi yang aman, basis data, cache, kendali versi, serta quality gate.
- Dalam studi kasus Kantin Multi-Tenant, modular monolith dapat menjadi pilihan awal yang proporsional bagi satu tim dengan target rilis tunggal.

## Istilah Penting

| Istilah | Makna ringkas |
|---|---|
| **SSR** | Pendekatan render halaman HTML di server |
| **SPA** | Aplikasi yang banyak mengelola pembaruan antarmuka di sisi klien |
| **Modular monolith** | Satu unit penerapan dengan modul-modul yang memiliki batas tanggung jawab jelas |
| **Microservices** | Sejumlah layanan mandiri yang diterapkan dan berkomunikasi melalui jaringan |
| **MVC** | Pola pemisahan Model, View, dan Controller |
| **Service container** | Mekanisme pengelolaan dan penyediaan dependensi objek |
| **Dependency injection** | Pemberian dependensi dari luar kelas yang membutuhkannya |
| **Cache hit ratio** | Proporsi permintaan yang berhasil dilayani oleh cache |
| **Quality gate** | Pemeriksaan otomatis atau terstandar sebelum kode digabungkan |

*Tabel 1.7. Istilah penting Bab 1.*

## Pertanyaan dan Latihan

**A. Pemahaman**

1. Jelaskan perbedaan utama SSR, SPA, dan SSR dengan interaktivitas komponen.
2. Mengapa SSR tidak dapat dianggap selalu lebih cepat daripada SPA?
3. Jelaskan perbedaan antara monolith dan modular monolith.
4. Apa fungsi `composer.lock` dalam kerja tim?
5. Mengapa cache tidak boleh diperlakukan sebagai satu-satunya sumber kebenaran untuk data transaksi?

**B. Penerapan**

1. Sebuah aplikasi katalog memiliki 30 halaman informasi, formulir pemesanan, dan beberapa interaksi sederhana pada keranjang. Pilih pendekatan rendering yang paling sesuai menurut Anda. Jelaskan alasan, manfaat, dan keterbatasannya.
2. Susun *checklist* penyiapan lingkungan untuk anggota tim baru yang akan bergabung pada proyek Laravel. Checklist harus mencakup perangkat, dependensi, konfigurasi, migrasi, dan pemeriksaan awal.
3. Buat konfigurasi `.env` lokal untuk aplikasi hipotetis bernama `TokoKampus`. Gunakan *placeholder* untuk seluruh nilai rahasia.

4. Dengan Persamaan 1.1, hitung percepatan untuk $p$ = 0,8 pada $N$ = 4 dan $N$ = 16. Hitung pula percepatan maksimum teoretisnya.
5. Dengan Persamaan 1.3, hitung $L$_eff jika $L$_cache = 2 ms, $L$_db = 30 ms, dan $h$ = 0,85. Bandingkan hasilnya dengan kondisi $h$ = 0,6.
6. Audit kutipan konfigurasi berikut dan sebutkan minimal tiga masalah beserta perbaikannya: `APP_ENV=production`, `APP_DEBUG=true`, `DB_PASSWORD=rahasia`, `CACHE_STORE=redis` tanpa layanan Redis yang berjalan, serta berkas `.env` yang ikut ter-commit ke repositori.

**C. Analisis**

1. Sebuah tim beranggotakan lima orang mengembangkan aplikasi kampus dengan modul akun, katalog, peminjaman, notifikasi, dan laporan. Aplikasi akan dirilis sekali pada akhir semester. Analisis apakah monolith, modular monolith, atau microservices paling sesuai. Gunakan sedikitnya empat kriteria dari Tabel 1.2.
2. Sebuah aplikasi menerima rata-rata 50 permintaan per detik dengan waktu penyelesaian rata-rata 0,08 detik. Gunakan Persamaan 1.2 untuk menghitung jumlah rata-rata permintaan dalam sistem. Jelaskan mengapa hasil tersebut belum cukup untuk menentukan jumlah koneksi basis data.
3. Tinjau Kode 1.4. Sebutkan dua risiko apabila kunci *health check* ditulis tanpa awalan dan tanpa masa berlaku. Usulkan perbaikannya.
4. Quality gate tim selalu lulus di komputer pengembang, tetapi banyak kesalahan muncul setelah kode digabungkan. Analisis tiga kemungkinan penyebab dan usulkan langkah perbaikannya.
5. Jelaskan kondisi yang dapat menjadi alasan kuat untuk memisahkan satu modul dari modular monolith menjadi layanan mandiri.
6. Setelah harga sebuah menu diperbarui oleh operator tenant, pelanggan masih melihat harga lama selama beberapa menit. Analisis kemungkinan penyebabnya dari sisi strategi cache, lalu usulkan dua perbaikan beserta konsekuensinya terhadap beban basis data.

## Daftar Pustaka


**Buku**

Evans, E. (2003). *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley.

Fowler, M. (2002). *Patterns of Enterprise Application Architecture*. Addison-Wesley.

Kleppmann, M. (2017). *Designing Data-Intensive Applications*. O'Reilly Media.

Newman, S. (2021). *Building Microservices* (2nd ed.). O'Reilly Media.

Richards, M., & Ford, N. (2020). *Fundamentals of Software Architecture: An Engineering Approach*. O'Reilly Media.

Stauffer, M. (2023). *Laravel: Up & Running* (3rd ed.). O'Reilly Media.

**Artikel Jurnal dan Prosiding**

Amdahl, G. M. (1967). Validity of the single processor approach to achieving large scale computing capabilities. *Proceedings of the AFIPS Spring Joint Computer Conference, 30*, 483–485. https://doi.org/10.1145/1465482.1465560

Little, J. D. C. (1961). A proof for the queuing formula: L = λW. *Operations Research, 9*(3), 383–387. https://doi.org/10.1287/opre.9.3.383

**Dokumentasi (Daring)**

Laravel LLC. (2026). *Laravel 13.x Documentation: The Service Container; Service Providers; Configuration*. Diakses 17 Agustus 2026 dari https://laravel.com/docs/13.x

Livewire. (2026). *Livewire Documentation*. Diakses 17 Agustus 2026 dari https://livewire.laravel.com/docs

Composer. (2026). *Composer Documentation: Versions and Constraints*. Diakses 17 Agustus 2026 dari https://getcomposer.org/doc

**Sumber Proyek (Dokumen Internal)**

Politeknik Negeri Banyuwangi. *SRS Aplikasi Kantin Multi-Tenant (ISO/IEC/IEEE 29148), versi 2 — keputusan arsitektur Fase 0*. Dokumen proyek.

Politeknik Negeri Banyuwangi. *Modul Praktikum Kantin Multi-Tenant Laravel 13 (Mentor Tutorial) — Modul 1: penyiapan lingkungan dan quality gate*. Dokumen praktikum.